

Apartado 1: Análisis de mejora progresiva en una web

Usando los navegadores Chrome o Firefox, elige varias webs de distintos tipos y comprueba si cumplen el principio de mejora progresiva. Comprueba si tienen y funciona la etiqueta **<noscript>** y su comportamiento si desactivas JavaScript en el navegador. Recomendación: busca webs o aplicaciones web variaditas (redes sociales, webs promocionales de empresas,

Revisa

1. Compara el resultado de **Ver Código Fuente** (HTML que envía el server) con lo que se muestra en el panel **Elements** de las DevTools (HTML ya modificado por JavaScript).
 - a. Si con Ver Código Fuente ya está todo el contenido real (textos, artículos, productos...) → la página viene renderizada desde el servidor (HTML tradicional, o SSR: Server-Side Rendering).
 - b. Si está casi vacío, pero luego en el panel Elements aparece el contenido → SPA: el HTML inicial está casi vacío y JavaScript construye el contenido después en el navegador.
 - c. También puede ser híbrida (*hidratada*, ni para ti ni para mí)
2. Con **JavaScript desactivado**:
 - a. ¿La página sigue mostrando contenido básico (texto, imágenes, navegación)? ¿O aparece en blanco/rota?
 - b. ¿Aparece algún mensaje de aviso? (Es decir, <noscript> funcionando)?
 - i. Compruébalo: en las DevTools, pestaña Elements, haz Ctrl+F dentro del panel y busca la etiqueta noscript en el HTML.
 - c. ¿Qué elementos dejan de funcionar? (menús desplegables, formularios de búsqueda, carruseles, imágenes que no cargan, botones que no responden).
 - d. Clasifica la web:
 - i. Sigue siendo usable razonablemente
 - ii. Parcialmente rota
 - iii. Totalmente inutilizable.
 - e. ¿Qué categorías o tipos de webs resisten mejor (respetan el principio de mejora progresiva)?
 - f. Elige las dos con resultados más alejados entre sí (la que mejor responde y la que peor responde).

Cómo desactivar JavaScript con las DevTools

Seguro que ya sabéis, pero para tenerlo de referencia:

En Chrome: Hay dos formas. La primera es más visual, dentro de las DevTools.

- **Command Menu de Chrome DevTools:**
 - Abre las DevTools: F12, Ctrl+Shift+I o clic derecho → **Inspeccionar**.
 - Con las DevTools abiertas, pulsa Ctrl+Shift+P. Se abre el **Command Menu**.

- Escribe *disable javascript* para filtrar y pulsa selecciona la opción **Disable JavaScript**.
- Actualiza la página. Debería cargarse sin ejecutar ningún script.
- Reactívalo con **Enable JavaScript**.
- **Ajustes de sitio (Chrome)**
 - Clic en el icono de candado/información a la izquierda de la URL.
 - Configuración de sitios o Site settings.
 - Busca *JavaScript* y bloquea el permiso (Block).
 - Actualiza la página.

En Firefox:

- Escribe `about:config` en la barra de direcciones y acepta el warning.
- Busca **javascript.enabled** y cámbialo a false con doble clic.
- Recarga la página.

Ejemplos propuestos para revisar:

- Airbnb
- Gmail
- Figma
- Aliexpress
- Google Maps
- Booking.com
- Skyscanner
- YouTube
- Trello en modo de ver tablero público
- Wikipedia
- Hacker News
- Stack Overflow
- MDN Web Docs (developer.mozilla.org)
- Craigslist
- La web del periódico que prefieras
- Amazon
- GitHub

Apartado 2: Investiga las cabeceras de seguridad de una web

Vamos a usar, nuevamente, Chrome o Firefox.

Ver las cabeceras de respuesta con DevTools

1. Abre las DevTools
2. Comprueba si la pestaña **Network** (Red) está vacía y recarga la página con las DevTools están, para que capture todas las peticiones.
3. En la lista de peticiones, haz clic en la primera fila (normalmente el documento HTML principal, con tipo document). Puedes filtrar el listado de Network con el nombre de dominio o con Doc (documento) para ver solo con la petición principal.
4. Se debería abrir un panel con varias pestañas: **Headers** (cabeceras), Preview, Response, etc.
5. Dentro de Headers, busca la sección "**Response Headers**".
6. Busca la línea **content-security-policy**.
 - a. Si aparece, la web tiene CSP activa.
 - b. Si no aparece, no la tiene por cabecera HTTP. En este caso, revisa si en la pestaña Elements hay una etiqueta **<meta http-equiv="Content-Security-Policy"...**
7. Si aparece, copia la parte de `script-src`.

Podéis usar los ejemplos propuestos de antes, otros que os gusten más o la web de alguna entidad bancaria sin identificación concreta.

Apartado 3: Evolucionando HolaMundo.js

Partimos del ejercicio hecho en clase (HolaMundo.js)

1. De alert() a modificar el DOM

Sustituye la línea `alert('Ola mundo dende DWCC')` por el siguiente código:

```
const saludar = () => {  
  
    document.querySelector('#mensaje').textContent = 'Ola mundo dende DWCC';  
  
}
```

Añade, en index.html, añadir una etiqueta de párrafo vacía, con id **mensaje**:

```
<p id="mensaje"></p>
```

Guarda y actualiza la página.

Pregunta: ¿por qué `alert()` "bloquea" la página hasta que se pulsa Aceptar y esto no?

2. De onclick en el HTML a addEventListener en el JS

Vamos a poner toda la lógica en JavaScript:

1. En *index.html*, quita el atributo *onclick* y añade un identificador al botón:

```
<button id="btnSaludar">Saludar</button>.
```

2. En *miScript.js*, añade al final:

```
document.querySelector('#btnSaludar').addEventListener('click',  
saludar);
```

Pregunta: ¿qué crees que pasaría si este código se ejecutase antes de que el botón exista en el HTML?

3. Convertir a módulo (*type="module"*) y comprobar el ámbito

Abre la consola de las DevTools e introduce *saludar()*. Comprueba lo que ocurre.

Cambia la etiqueta en *index.html*:

```
<script type="module" src="./scripts/miScript.js"></script>
```

Recarga con Live Server y comprueba si el botón sigue funcionando igual.

Abre la consola de las DevTools e introduce *saludar()*

- Sin *type="module"*: *saludar* existe en el ámbito global y la consola ejecuta la función sin problema.
- Con *type="module"*: la consola debe devolver un error del tipo *saludar is not defined*.

4. Comprobar defer vs async con dos scripts

Crear un segundo archivo *scripts/segundo.js* con:

```
console.log('Segundo script ejecutado');
```

En el *<head>* de *index.html*, prueba las siguientes dos configuraciones (una detrás de otra, comentando la que no se usa):

```
<!-- Prueba A -->  
<script src="./scripts/miScript.js" defer></script>  
<script src="./scripts/segundo.js" defer></script>
```

```
<!-- Prueba B -->  
<script src="./scripts/miScript.js" async></script>  
<script src="./scripts/segundo.js" async></script>
```

Con la pestaña Console abierta, recarga varias veces con cada configuración y anota: con defer, ¿siempre se ejecutan en el mismo orden? ¿Y con async?

5. Añadir el <noscript>

Dentro del <body>, antes del botón, añade:

```
<noscript>  
  <p>Esta página necesita JavaScript para que no se produzca una fusión  
del núcleo.</p>  
</noscript>
```

Comprueba (desactivando JS desde las DevTools) que el mensaje aparece solo cuando JS está desactivado, y que el botón/mensaje de saludo desaparecen.

6. Comprobar que el código nunca está oculto

Aunque el JS esté en un archivo externo:

Con Live Server activo, vete a la URL del script directamente en el navegador (será algo como `http://127.0.0.1:5500/scripts/miScript.js`)

Pregunta: ¿Qué se ve?